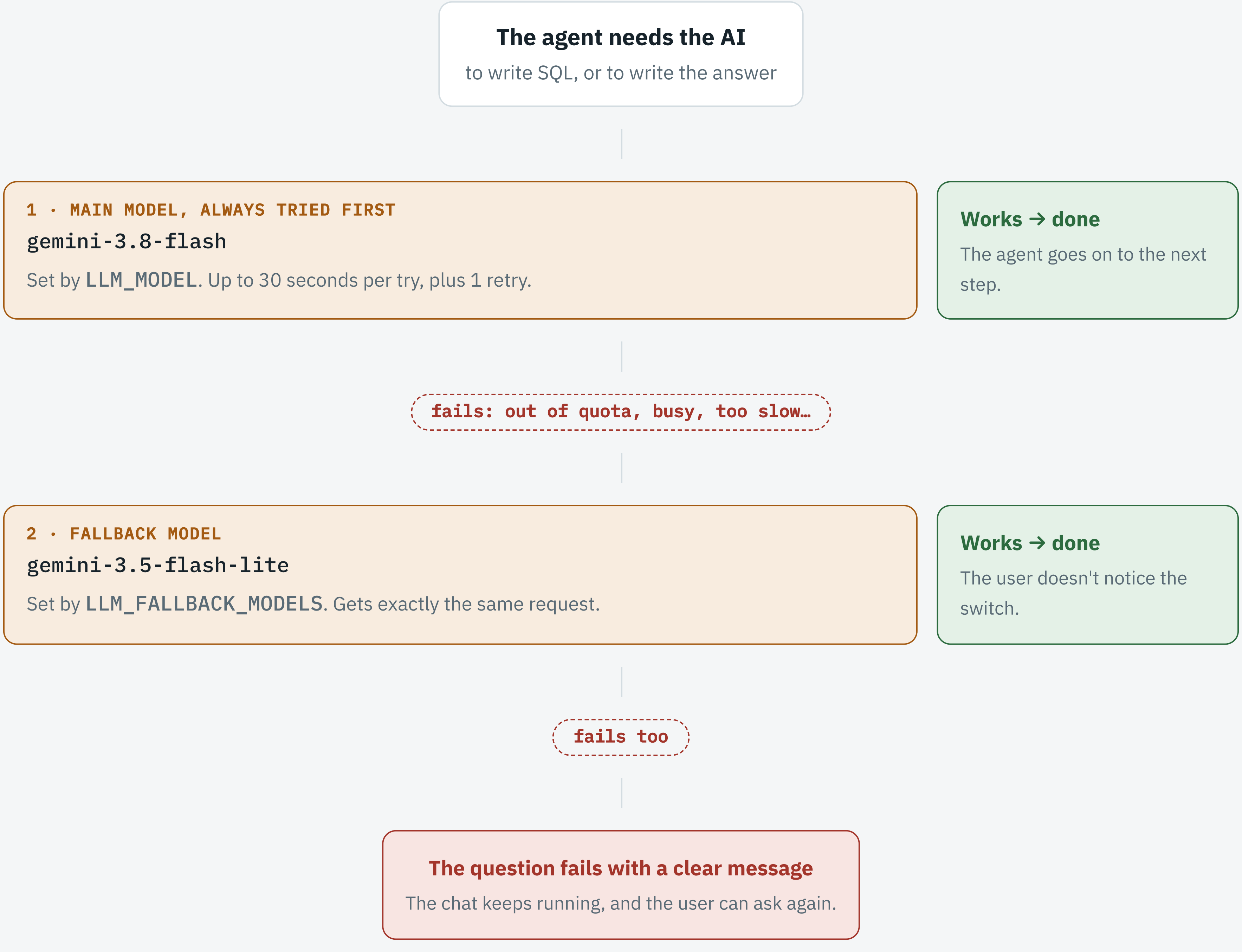
RETAIL DATA AGENT · RESILIENCE

When a Gemini model fails, the next one takes over

Every call to the AI goes to the main Gemini model first. If that model is out of quota, busy or too slow, the same request goes to a second Gemini model. The user still gets an answer, and if every model fails they get a clear message while the chat keeps running.



The agent needs the AI
to write SQL, or to write the answer

1 · MAIN MODEL, ALWAYS TRIED FIRST
`gemini-3.8-flash`
Set by `LLM_MODEL`. Up to 30 seconds per try, plus 1 retry.

Works → done
The agent goes on to the next step.

`fails: out of quota, busy, too slow…`

2 · FALLBACK MODEL
`gemini-3.5-flash-lite`
Set by `LLM_FALLBACK_MODELS`. Gets exactly the same request.

Works → done
The user doesn't notice the switch.

`fails too`

The question fails with a clear message
The chat keeps running, and the user can ask again.

What makes it switch

Any failed call switches to the next model. These are the failures it is there for:

429
Out of quota: the free tier allows 20 requests a day per model

503
The model is busy ("high demand")

504
Took too long and timed out

Network
The connection dropped

A mistake in our own setup, like a wrong API key, also tries the next model, then fails with that model's error. That costs one wasted call and nothing more.

It decides at every call, not once per question

One question takes several AI calls: usually one to write the SQL and one to write the answer. Each call starts with the main model again, so the agent goes back to it as soon as it recovers.

A real run (31 October 2026)

"How many products does the brand 'calvin klein' have?"

1. **Call 1, main model:** `gemini-3.6-flash` wrote the SQL in 3.8 s.

```
SELECT COUNT(*) AS product_count FROM `…products` WHERE LOWER(brand) = 'calvin klein'
```

2. **BigQuery** ran it in 2.4 s and returned 1 row.

3. **Call 2, main model:** out of quota after its 20 requests for the day.

```
429 RESOURCE_EXHAUSTED: Generate content requests per day limit exceeded on Free Tier
```

4. **Call 2, fallback model:** `gemini-3.5-flash-lite` took over at once with the same request...

5. ...but Gemini was slow that day, so it timed out too.

```
504 DEADLINE_EXCEEDED
```

6. **The question failed with a clear message**, and the chat kept running. With a healthy fallback model, call 2 would have written the answer.

The limits

×2
questions on a free key
Each Gemini model has its own quota of 20 requests a day. At 2–5 calls per question that is 4–10 questions per model, so a second model about doubles it.

0
extra calls on the budget
A question may use at most 5 AI calls. A fallback try replaces a call that failed, so it doesn't use one up.

Gemini
only
Fallback models use the same provider as the main model, so Gemini never switches to Claude or OpenAI.

Why only Gemini models

The client asked for this, and their reviewers test with a free Gemini key only. Claude, OpenAI and Azure OpenAI can still be chosen in config; they are just switched off by default.

CHOSEN · GEMINI TO GEMINI

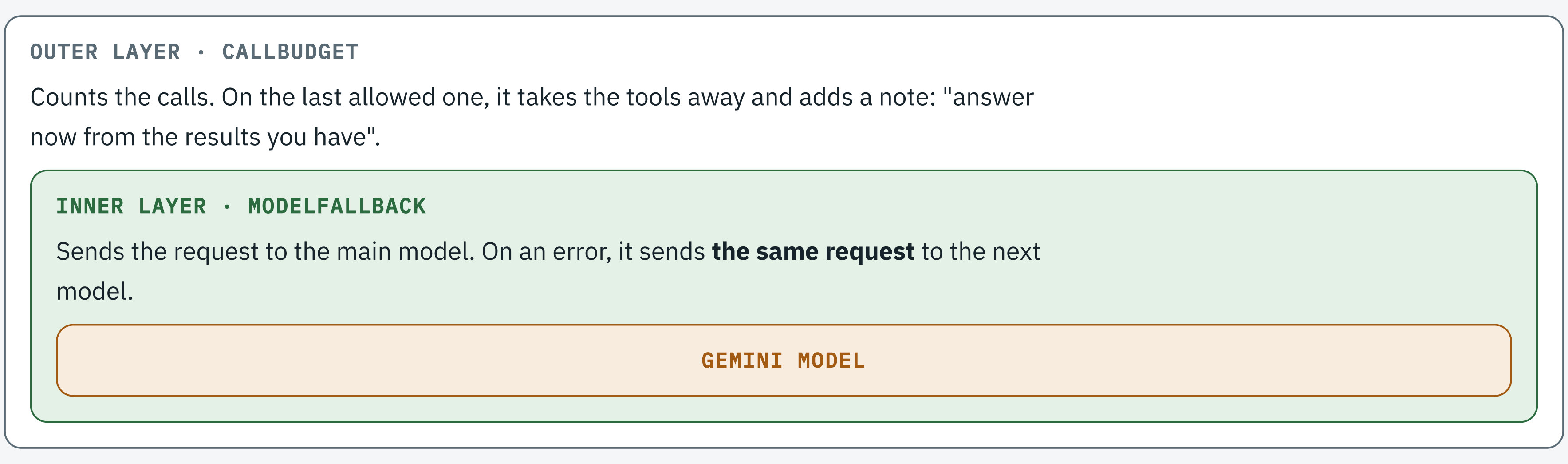
- Works with one free key; reviewers need nothing else.
- Every model reads the same prompts and tools the same way.
- Each model's quota adds to the total.
- **Weak spot:** a Gemini-wide outage takes down every model at once, as on 31 October, when both timed out.

NOT CHOSEN · SWITCH TO ANOTHER PROVIDER

- Survives a full Gemini outage.
- Needs a second paid key, which reviewers don't have.
- Answers can differ in style and SQL habits mid-conversation.
- In production it's a small change: let each fallback model name its own provider (e.g. `anthropic:claude-haiku-5-5`).

How it plugs in: two layers around every call

The agent doesn't call the model directly. Each call passes through *middleware*: layers of code that see the request on its way in and the reply or error on its way out. The first layer in the list is the outermost, so it sees the request first.



OUTER LAYER · CALLBUDGET
Counts the calls. On the last allowed one, it takes the tools away and adds a note: "answer now from the results you have".

INNER LAYER · MODELFALLBACK
Sends the request to the main model. On an error, it sends **the same request** to the next model.

GEMINI MODEL

Does the order matter? Not here, but it's still deliberate

The risk to watch is the **last call**: there the model must get no tools, so it answers instead of running another query. Both orders keep that rule, because a layer acts on every request that passes through it:

CHOSEN · CALLBUDGET OUTSIDE

```
CallBudget removes the tools   (once)
  ↳ Fallback: main model     ❌ 429
  ↳ Fallback: next model     ✅ gets the request
                                with no tools
```

The request is changed once, and every model after it gets the changed request. It reads top to bottom in the same order things happen.

ALSO WORKS · FALLBACK OUTSIDE

```
Fallback: try main model
  ↳ CallBudget removes the tools
  ↳ main model               ❌ 429
Fallback: try next model
  ↳ CallBudget removes the tools again
  ↳ next model               ✅ no tools
```

The retry goes back through CallBudget, so it changes the request a second time. Same result, more work, and harder to follow.

Checked both ways: with each order, the fallback model on the last call got no tools and saw the "answer now" note. A test keeps it that way.

Middleware is also why this took only a few lines. LangChain already ships `ModelFallbackMiddleware`, so switching models is one more layer in the list. It isn't a change to the agent loop, the tools or the gateway.

Settings

SETTING	DEFAULT	WHAT IT DOES
`LLM_MODEL`	`gemini-3.8-flash`	The main model, tried first on every call.
`LLM_FALLBACK_MODELS`	`["gemini-3.5-flash-lite"]`	Models tried in order when a call fails. `[]` turns the fallback off.
`LLM_TIMEOUT_SECONDS`	`30`	How long one try may take before it counts as failed.
`LLM_MAX_RETRIES`	`1`	Retries on the same model before switching.

> **The rule in one sentence**
> Always try the main model first, and when it fails, give the same request to the next Gemini model on the list, without spending any of the question's 5 calls.

For engineers: the switch is LangChain's `ModelFallbackMiddleware`, added in `build_agent` in `src/retail_agent/agent.py`. It sits inside `CallBudget`, so `CallBudget` changes the request once and every model gets it as changed (including the last call, which has no tools). The models are built by `build_model(settings, model_name)` in `src/retail_agent/llm.py` with the provider from `LLM_PROVIDER`. Each call and its model name are logged by `LlmCallLogger`, so a switch shows up in `logs/retail-agent.log` as a failed call followed by a call to the next model.

Tests: `test_a_failing_model_falls_back_to_the_next_model_which_keeps_the_tools` and `test_the_question_fails_when_every_model_fails` and `test_a_fallback_model_on_the_last_allowed_call_gets_no_tools` in `tests/test_turn.py`.